

Compiler project schedule

A proposed 15-week plan-driven development schedule

Planning assumptions. A team of 3-4 students, each contributing approximately 8-10 hours per week. Build a compiler for a small, statically typed teaching language targeting a stack-based virtual machine with a supplied runtime. Weeks are relative to project kickoff; this is a planning template, not a dated course calendar.

Scope and development model

Core scope: integer and boolean expressions, variables, functions, conditionals, lexical analysis, parsing, semantic checks, an intermediate representation, code generation, diagnostics, and a command-line driver. Closures, advanced optimization, native machine code, and a new runtime are excluded unless approved during requirements review.

Baseline the requirements and architecture before major implementation. Complete each phase against written exit criteria. Prepare tests early, run them throughout development, and use formal reviews to authorize movement to the next phase.

Phase schedule and approval gates

Weeks	Phase and principal work	Deliverable / exit gate
1-2	Requirements and feasibility: define syntax, types, evaluation rules, target, diagnostics, and acceptance examples. Prototype the riskiest target interface.	G1 - Requirements baseline: language specification, scope exclusions, feasibility result, acceptance plan, and effort estimate approved.
3-4	Architecture and detailed design: define tokens, grammar, AST, symbol tables, type rules, IR, calling convention, and module interfaces. Design test fixtures.	G2 - Design baseline: reviewed design, interface contracts, module assignments, traceability matrix, and updated schedule approved.
5-6	Front end: implement lexer, parser, AST construction, source locations, and syntax diagnostics.	G3 - Front-end review: valid and invalid syntax tests pass; AST output and diagnostics match the specification.
7-8	Semantic analysis: implement scope resolution, type checking, function checks, and typed AST or IR construction.	G4 - Semantic review: required positive and negative semantic tests pass; invalid programs are rejected before code generation.
9-10	Back end: implement lowering, stack-VM code emission, control flow, function calls, and driver integration.	G5 - Feature complete: all required constructs compile and execute in representative end-to-end tests.

Integration, acceptance, and delivery

Weeks	Phase and principal work	Deliverable / exit gate
11-12	System integration and verification: run the full regression suite, test interactions between features, compare execution with expected results, and fix defects.	G6 - Verification complete: required system tests pass; no open critical correctness defects; test report reviewed.
13	Acceptance: run the agreed acceptance suite from a clean checkout. Review user documentation and installation instructions.	G7 - Release candidate: instructor/client accepts required behavior; remaining minor defects are documented.
14	Contingency and stabilization: resolve review findings, improve diagnostics and documentation, and repeat affected tests. No new features.	G8 - Release readiness: regression and acceptance suites pass; release checklist approved. Keep this week as explicit schedule reserve.
15	Delivery and project closure: demonstrate the compiler, package the release, hand over documentation, and review estimates versus actual effort.	G9 - Final delivery: tagged source, reproducible build, sample programs, test results, user guide, and final report handed over.

Responsibilities and weekly control

Suggested roles: project/design lead; front-end and semantic lead; back-end lead; verification and release lead. With three people, combine project and release coordination. Assign a reviewer other than the author for each major deliverable.

Weekly cadence: review completed work, remaining effort, defects, dependencies, and risks. Update the schedule and test evidence. At each gate, the team and instructor/client record approval or specific rework actions; unresolved gate criteria delay dependent work.

Dependencies and change control

Main dependency chain: language specification -> AST and interfaces -> semantic analysis -> lowering and code generation -> system verification -> acceptance. Test design and documentation proceed alongside the relevant phase. Back-end fixtures may use hand-built IR after G2, while integration waits for the agreed interfaces.

After G1, record each proposed scope change with its reason, affected requirements, effort, test impact, and schedule impact. The project lead and instructor/client approve or defer it. Revise the baseline explicitly; use the contingency week for defects and overruns, not unapproved features.

Completion criteria and risk response

The release must build from a clean checkout, pass all mandatory acceptance and regression tests, execute the documented examples correctly, and report invalid programs according to the specification. Document any remaining minor limitations. Set measurable performance requirements in weeks 1-2 only if the project needs them.

Resolve grammar ambiguity before G2; verify function-call and stack behavior with small fixtures before G5. If a phase slips, reassess remaining effort immediately and defer optional scope by approval. Preserve the required language features and acceptance work; rebaseline the delivery date if the reserve is insufficient.